PROCURA

Demo Walkthrough with Screenshots

An OS-inspired task scheduler with a self-healing, RAG-grounded AI layer

Screens covered	Dashboard, Processes, Scheduler (6 algorithms), Error Monitor, AI Assistant, API docs, terminal dashboard, benchmark, test suite
Figures	21 screenshots, all captured from the running system
Captured	19 October 2026
Repository	github.com/harshitchhabi/procura

Contents

Contents 2
1. How these screenshots were produced 3
1.1 Environment 3
1.2 Reproducing the walkthrough 3
2. Demo script for reviewers 4
3. Tasks as processes 5
4. CPU scheduling — six algorithms on the same seven tasks 9
5. Resource contention and self-healing 16
6. The AI assistant 21
7. API, terminal dashboard, benchmark and tests 23
8. Observations and known limitations visible in the demo 26

1. How these screenshots were produced

Every screenshot in this document was captured automatically from the running application, not mocked up or edited. The web screens were captured by a Playwright script driving a headless Chromium browser (version 156) through the real user interface at a 1440 × 900 viewport, rendered at 1.5× scale for print clarity. The three terminal screens were rendered by the same rich library the terminal dashboard uses and rasterised in Chromium; the only post-processing applied to any image is cropping the white margin around those three terminal images.

Each capture run started from a clean state: the database was deleted and reseeded with the six demo tasks from seed_data.py, the backend (FastAPI) and frontend (Vite) were restarted, and the script then performed the walkthrough below in order. The AI answers come from a live call to Groq's openai/gpt-oss-20b model; because language-model output varies between runs, the exact wording in Figures 14, 16 and 17 will differ if the walkthrough is repeated, while every other number is deterministic.

1.1 Environment

Item	Value
Operating system	Windows 11
Backend	Python 3.13, FastAPI + Uvicorn, SQLite — served on port 8010 (port 8000 was occupied by another program, so the configurable VITE_API_URL / PROCURA_API_URL settings were used)
Frontend	React 18 + Vite dev server on port 5173
Browser automation	Playwright with headless Chromium 156
LLM provider	Groq, model openai/gpt-oss-20b (optional; the app also runs fully offline)
Test suite at capture time	82 tests, all passing

Table 1. Capture environment.

1.2 Reproducing the walkthrough

- cd backend && python seed_data.py && uvicorn main:app --port 8010
- cd frontend && VITE_API_URL=http://localhost:8010 npm run dev
- Open http://localhost:5173 and follow the demo script in Section 2.

2. Demo script for reviewers

The figures follow a single storyline that can also be presented live in about ten minutes. Each step names the screen, the action, and the point the step demonstrates.

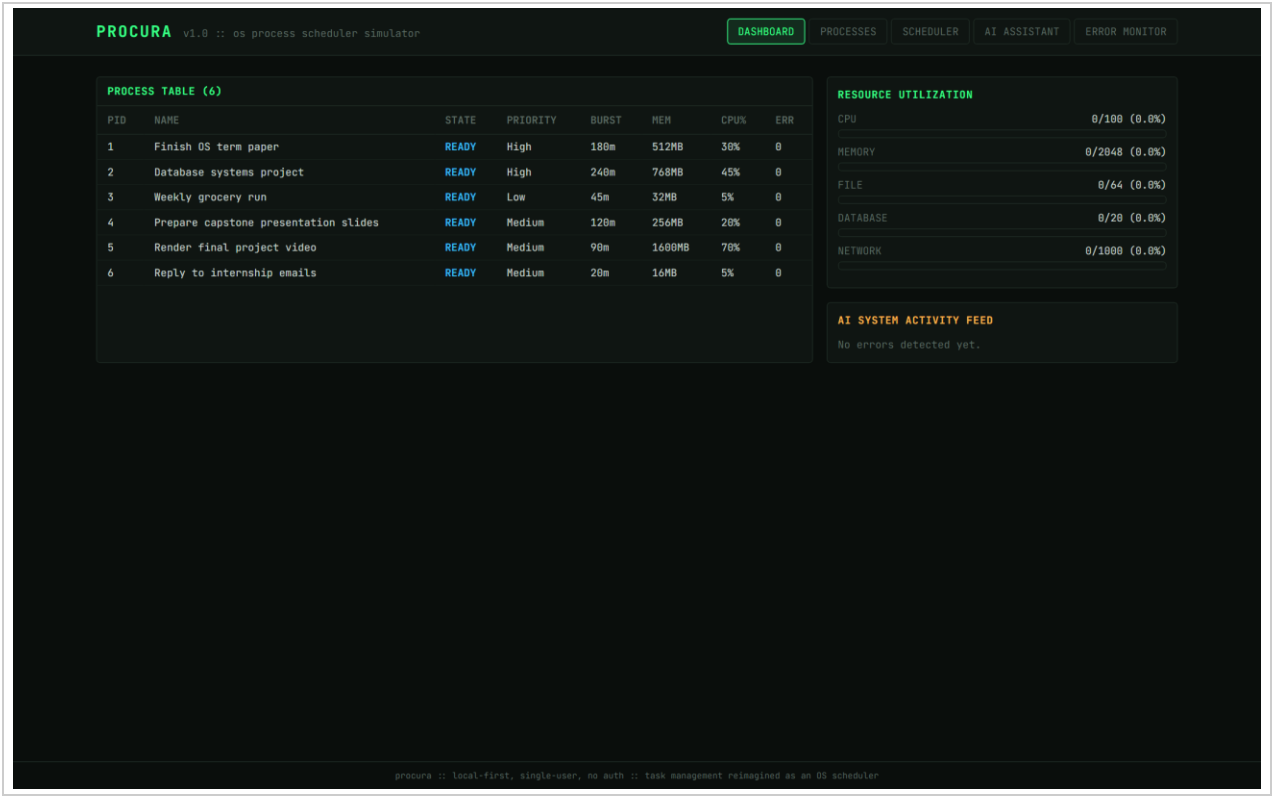
#	Screen	Action	What it demonstrates
1	Dashboard	Open the app	Tasks modelled as OS processes; live resource view
2	Processes	Create a task	Admission NEW → READY; state-machine-driven actions
3	Scheduler	Run all six algorithms	FCFS, SJF, Priority, Round Robin, EEVDF, MLFQ side by side on identical tasks
4	Dashboard	Apply memory pressure	A low-priority task hogs memory; 248 MB left
5	Error Monitor	RUN DETECT NOW	Detect → diagnose → recover, with three distinct, justified recovery outcomes
6	AI Assistant	Ask about P5	Answer grounded in recorded incident facts
7	AI Assistant	Ask about EEVDF	Answer grounded in the knowledge base
8	Terminal	procura_top, benchmark, tests	Same live data in a terminal; reproducible evaluation; 82 passing tests

Table 2. Ten-minute demo script.

3. Tasks as processes

Procura models every task as an operating-system process with a PID, a priority, a burst time (estimated minutes of work), memory and CPU requirements, an error counter, and a lifecycle state enforced by a state machine.

Figure 1. Dashboard on first load

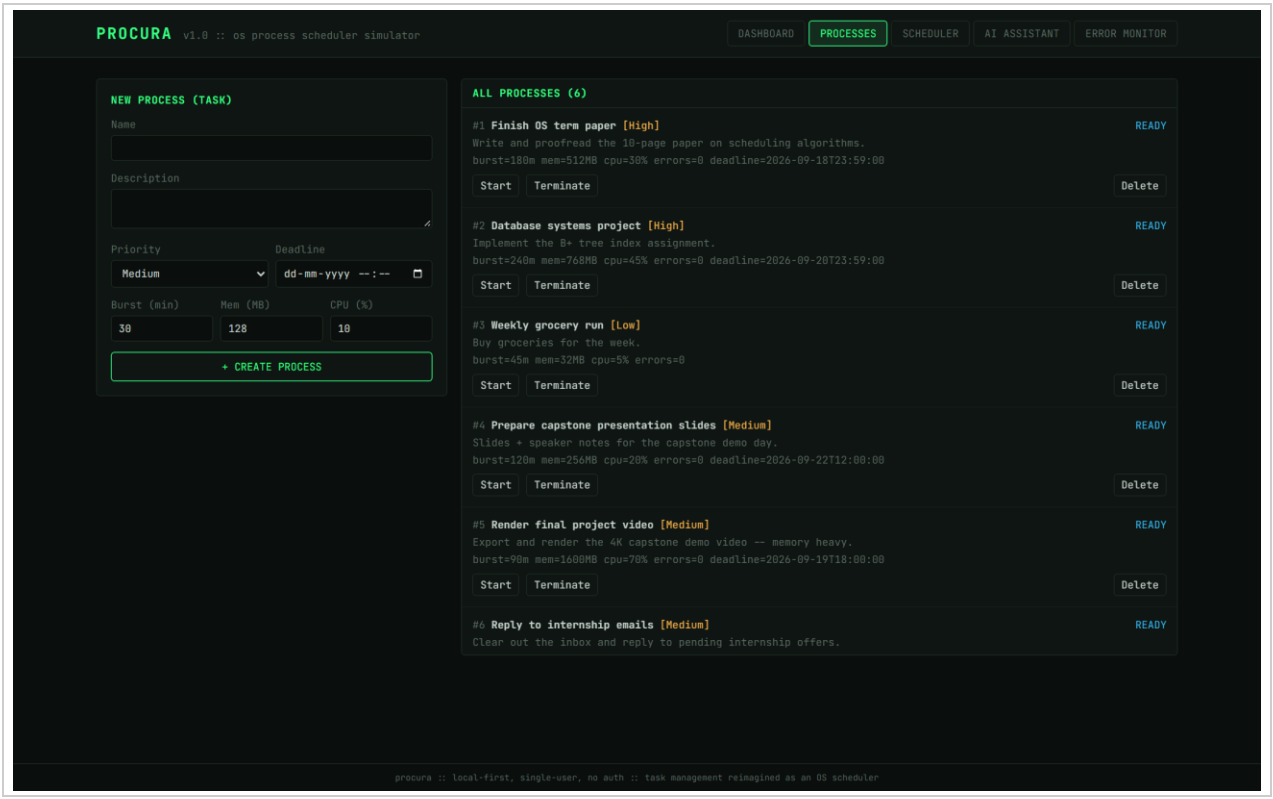


The dashboard combines the live process table, utilisation of the five simulated resource pools (CPU, memory, file handles, database connections, network bandwidth), and the AI SYSTEM activity feed.

What to notice

- All six seeded tasks are READY: the Process Manager admits each new task with the transition NEW → READY.
- Every resource pool is at 0% and the AI feed is empty, because no task has yet requested resources or failed.
- The memory pool's capacity of 2048 MB is the constraint exploited later in the demo.

Figure 2. Processes screen

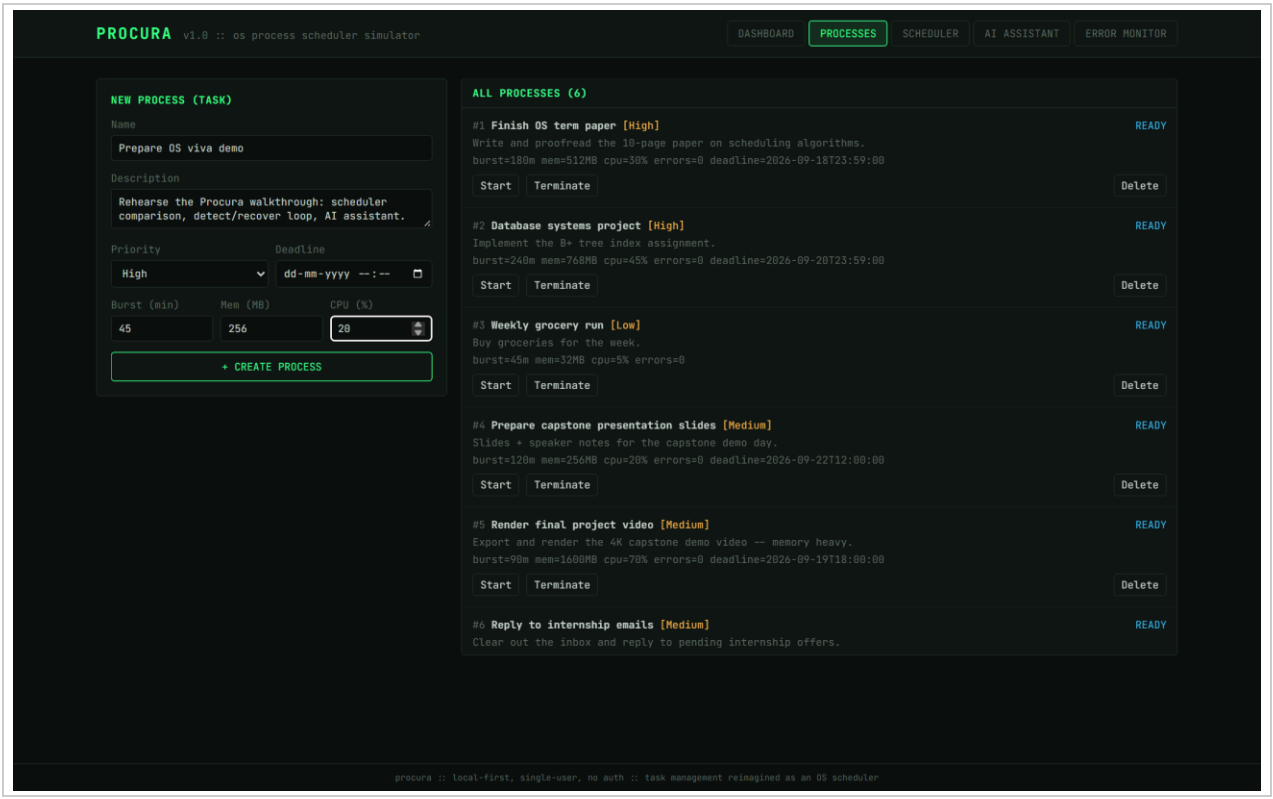


All tasks with their attributes and the lifecycle actions currently available for each.

What to notice

- The action buttons are generated from the state machine. A READY task offers only Start and Terminate — the two transitions the state machine permits from READY. A RUNNING task would instead offer Pause, Complete and Terminate, and a BLOCKED task Resume and Terminate.
- The backend enforces the same rules independently: an illegal transition is rejected with HTTP 409, so the UI cannot be bypassed.

Figure 3. Creating a new task

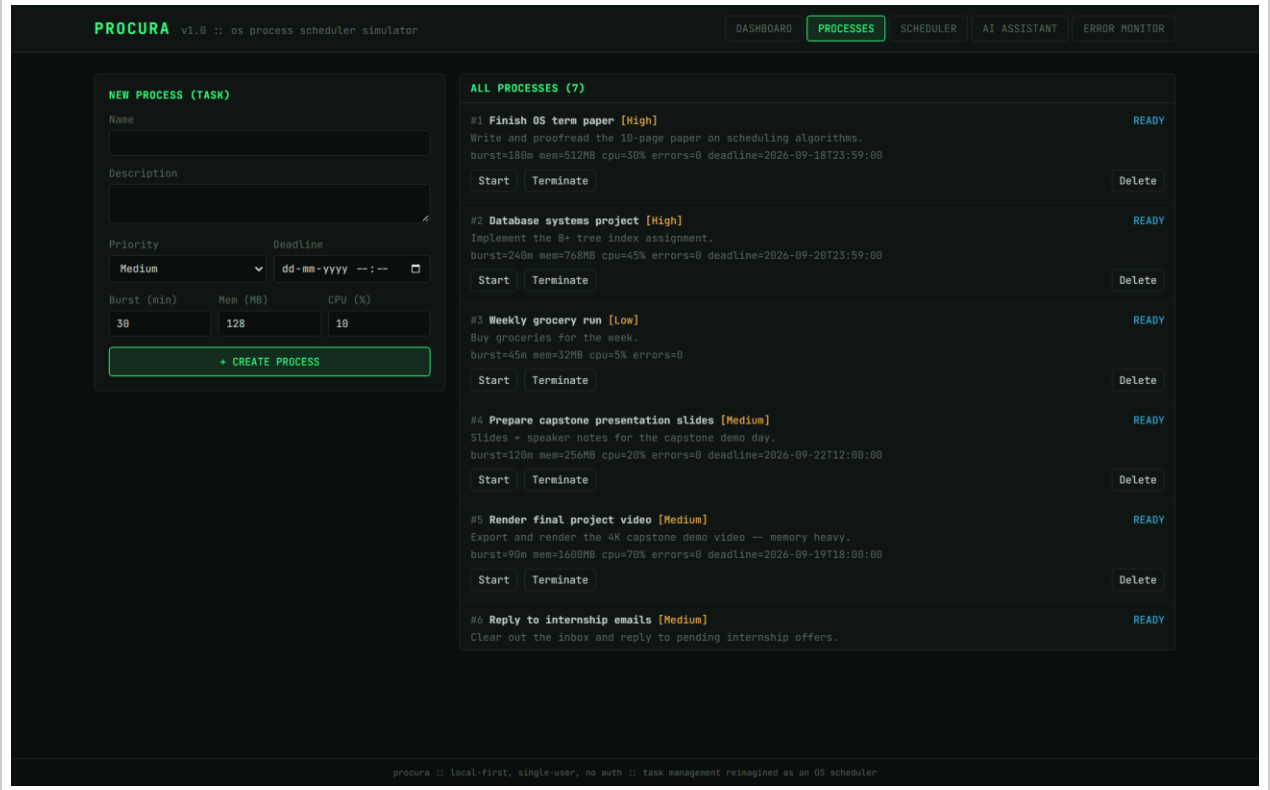


The form for a new task, filled in for 'Prepare OS viva demo'.

What to notice

- Each field maps to a process attribute used by the scheduler: priority feeds the effective-priority score, burst time drives SJF and the Gantt chart, and memory and CPU requirements feed the resource manager.
- The deadline is optional; a task without one contributes no deadline urgency to its priority score (this matters in Figure 7).

Figure 4. Task admitted as PID 7



After submission the new task appears as PID 7 in the READY state.

What to notice

- PID assignment and admission happen server-side, and the record is persisted in SQLite, so it survives a backend restart.

4. CPU scheduling — six algorithms on the same seven tasks

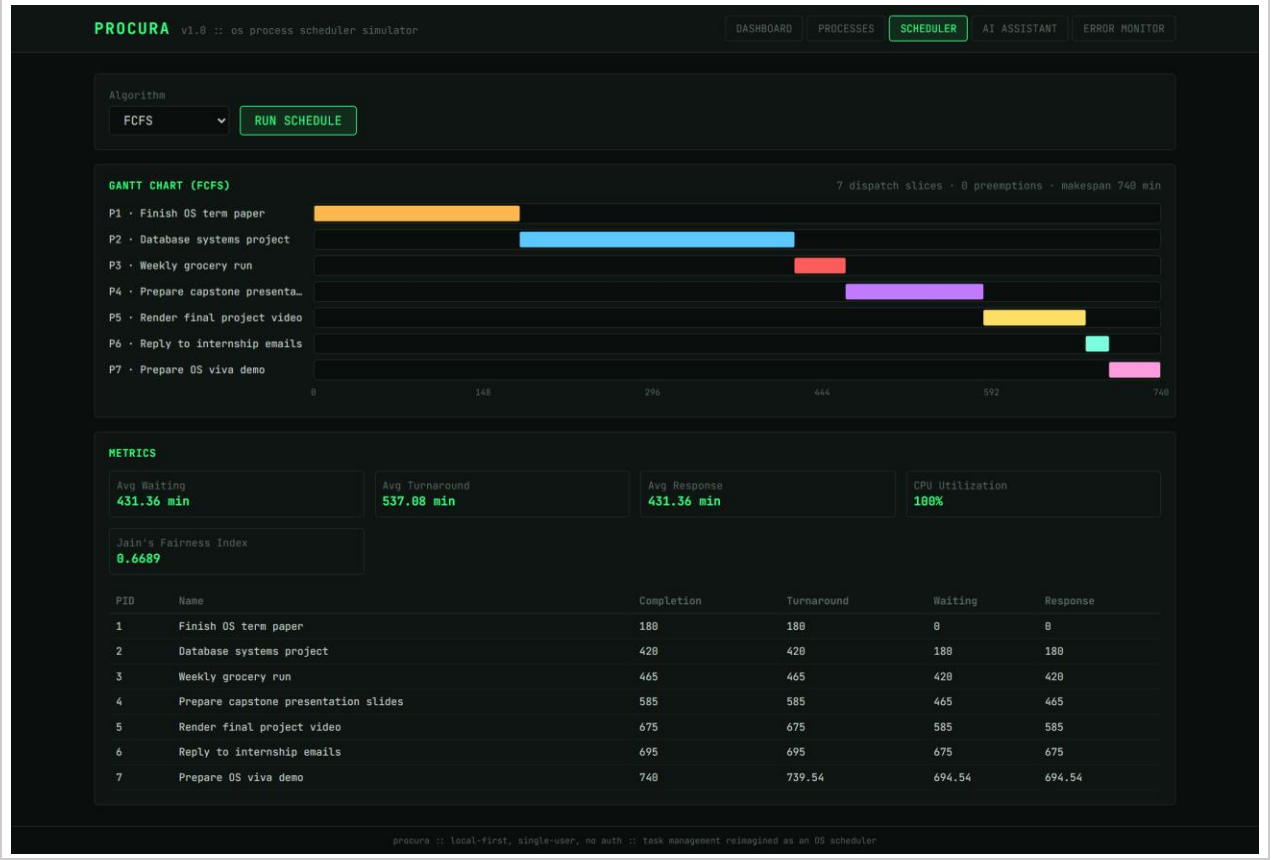
The Scheduler screen runs any of six policies over the current READY tasks and shows a Gantt chart (one lane per task, so preemption is visible as repeated slices), aggregate metrics including Jain's Fairness Index, and per-task completion, turnaround, waiting and response times. Table 3 summarises the six runs captured in Figures 5–10.

Algorithm	Slices	Preemptions	Avg wait (min)	Avg turnaround	Avg response	CPU util.	Jain's index
FCFS	7	0	431.37	537.09	431.37	100%	0.6689
SJF	7	0	173.51	279.23	173.51	100%	0.6689
Priority	7	0	436.37	542.09	436.37	100%	0.6689
Round Robin	187	180	348.94	454.66	11.94	100%	0.6689
EEVDF	187	180	440.23	545.94	17.09	100%	0.6689
MLFQ	110	103	368.37	474.09	11.94	100%	0.6689

Table 3. Live-demo comparison (seven tasks, makespan 740 minutes for every algorithm). Priority values are as shown in Figure 7; the others were recomputed from the final database and match the screenshots to within 0.01.

Reading the table. All six algorithms keep the CPU busy whenever a task is waiting, so they share the same makespan and 100% utilisation; they differ in who waits and for how long. SJF minimises average waiting time, as theory predicts for non-preemptive policies. Round Robin and MLFQ minimise response time — every task starts within about 12 minutes. Jain's index is identical for all six because every algorithm eventually gives every task exactly its burst time; see Section 8.

Figure 5. FCFS — First-Come, First-Served

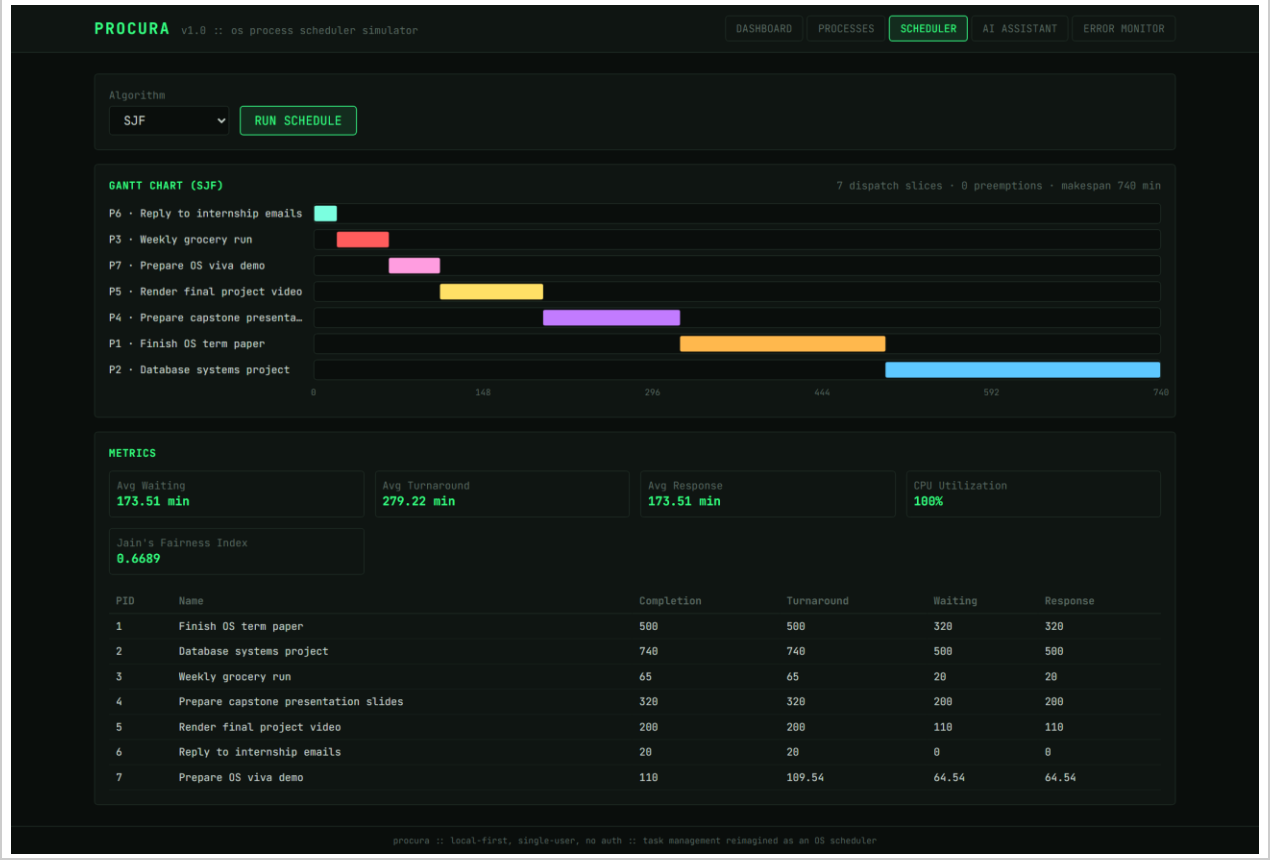


Tasks run once, start to finish, in arrival order.

What to notice

- Seven dispatch slices and zero preemptions: each lane has a single bar.
- Because nothing is preempted, response time equals waiting time (431.4 minutes on average).
- The 180-minute term paper and 240-minute database project at the head of the queue delay every shorter task behind them — the convoy effect.

Figure 6. SJF — Shortest Job First

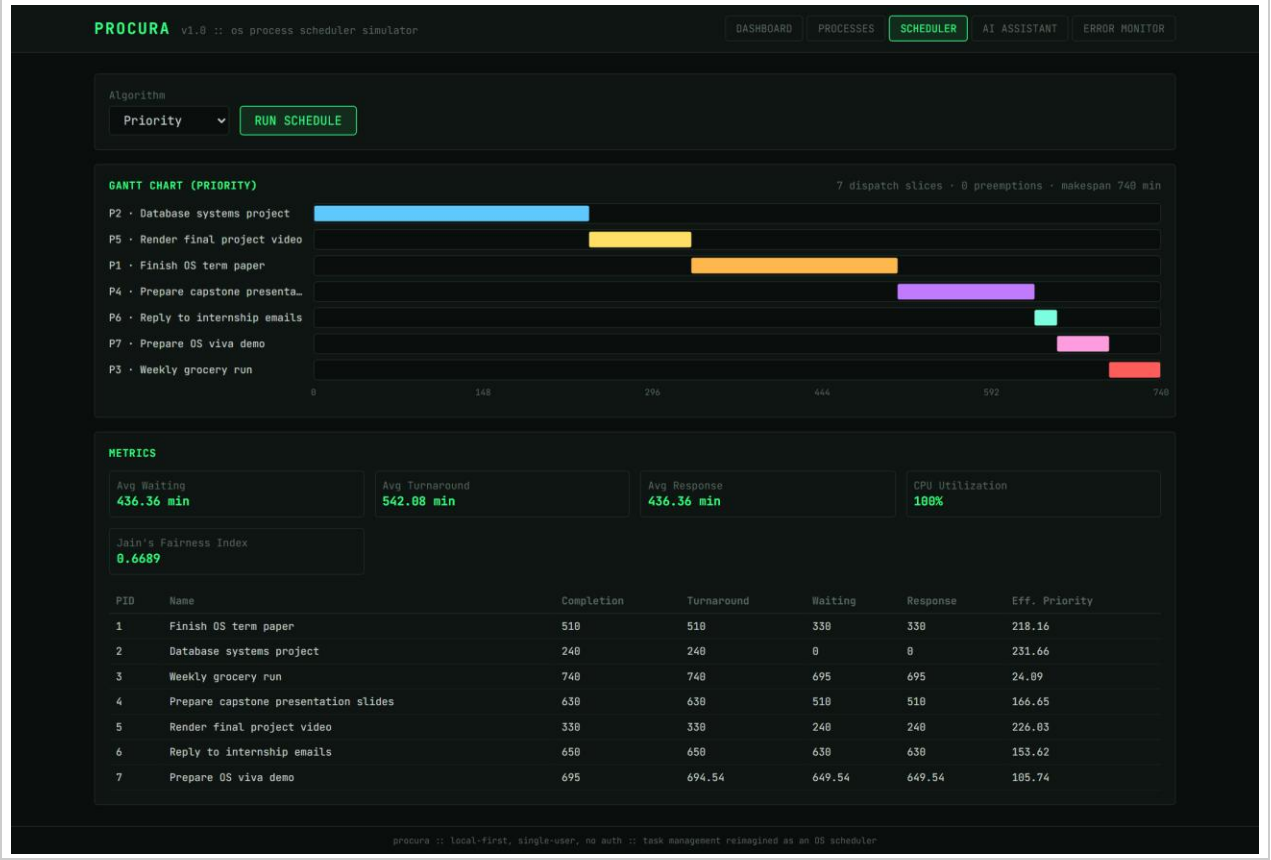


Among the tasks that have arrived, the shortest runs next.

What to notice

- P6 (20 minutes) runs first and P2 (240 minutes) last.
- SJF gives the lowest average waiting time of the six algorithms (173.5 minutes, versus 431.4 for FCFS).
- The cost falls on long tasks: P2 waits 500 minutes. This is the starvation risk SJF carries for long work.
- The chart starts at time 0 with no idle gap: SJF chooses only among tasks that have already arrived (an earlier version idled the CPU waiting for a late, shorter task; that defect was fixed and is covered by a regression test).

Figure 7. Priority — effective-priority scheduling

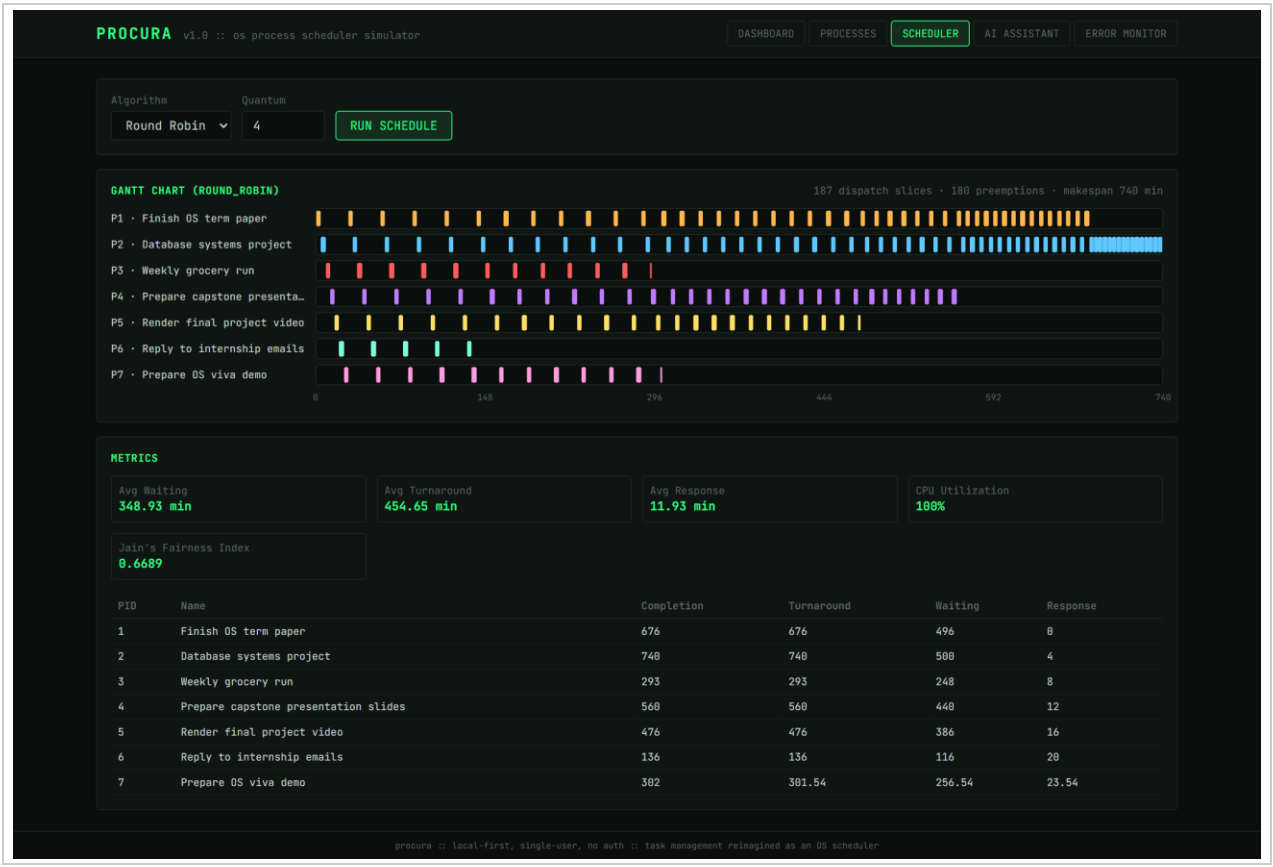


Tasks are ordered by a computed effective-priority score (last column), not by the raw High/Medium/Low label.

What to notice

- The score adds user priority (High 90, Medium 50, Low 20), deadline urgency (rising to 100 as a deadline passes), waiting-time aging, a resource-requirement term, and a Slurm-style fair-share penalty.
- P5 (Medium) runs before P1 (High): both deadlines have already passed, so both receive the maximum urgency score, and P5's 1600 MB / 70% CPU requirement adds a larger resource term (225.9 versus 218.0).
- P7 is labelled High but ranks sixth (105.7) because it has no deadline; P3, Low with no deadline, runs last (24.0).
- This is a design choice worth discussing in review: the weighting decides what the user is told to work on next.

Figure 8. Round Robin (quantum = 4 minutes)

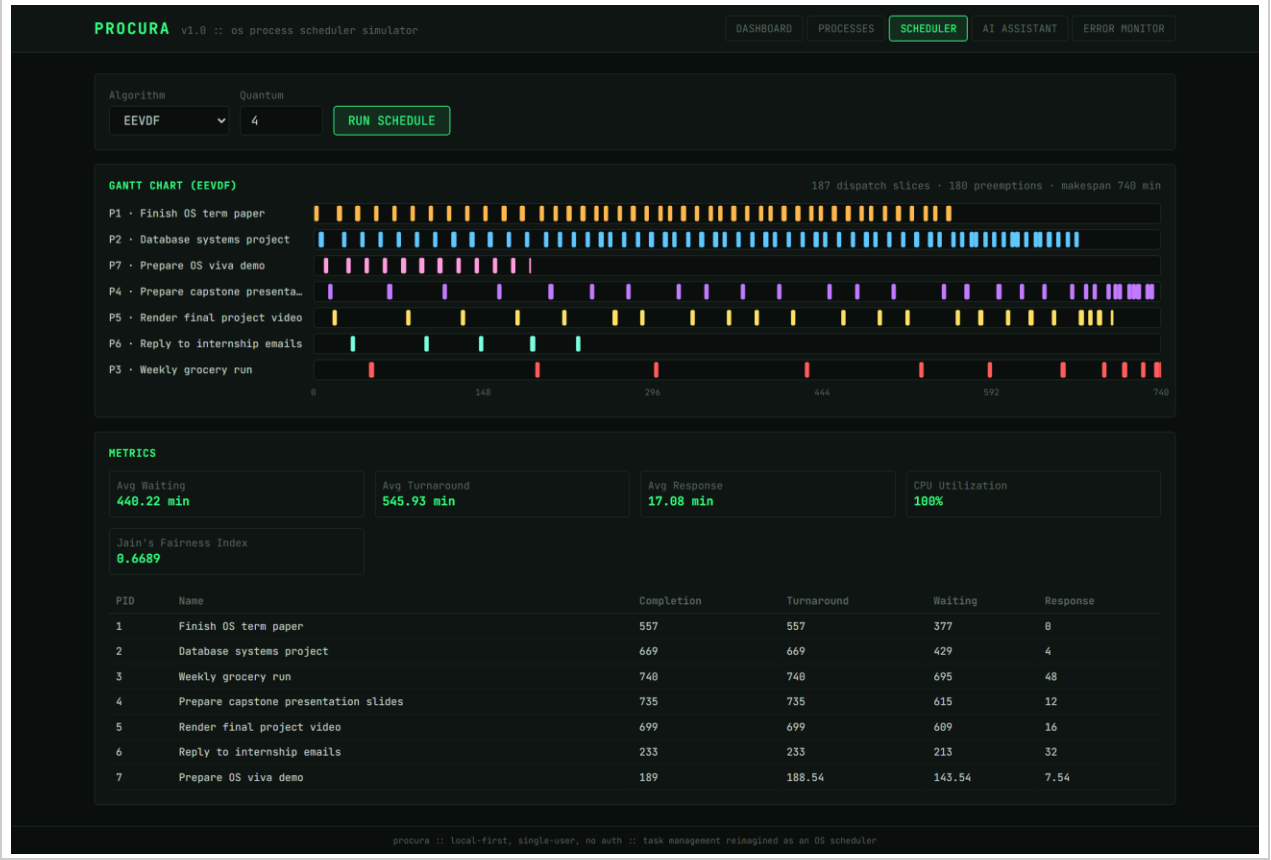


Every READY task receives a 4-minute turn in rotation.

What to notice

- 187 dispatch slices and 180 preemptions: each lane is a dense row of short slices.
- Average response time falls to 11.9 minutes — every task starts almost immediately — at the cost of a higher average waiting time than SJF, because long tasks are interleaved with short ones until the end.

Figure 9. EEVDF — Earliest Eligible Virtual Deadline First

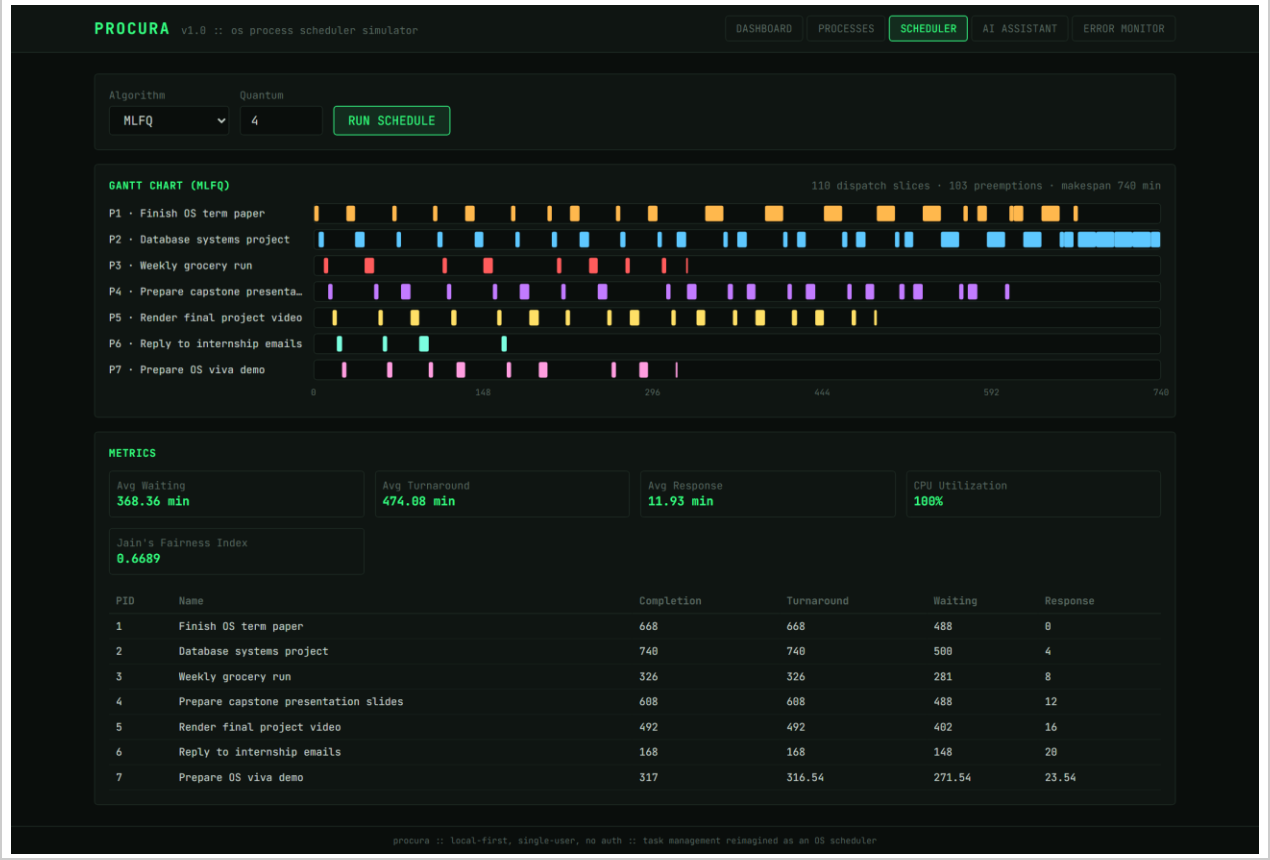


Linux's default scheduler since kernel 6.6, implemented with weights derived from priority, lag tracking, and a min-heap of virtual deadlines.

What to notice

- The same 187 slices as Round Robin, but no longer equal: High-priority P1 and P2 are dispatched far more often than Low-priority P3, whose slices are sparse.
- P3 is never starved — its lag accumulates until it becomes eligible — and it still completes.
- Response time reflects the weighting: 0 minutes for P1 but about 48 minutes for P3 (17.1 on average).

Figure 10. MLFQ — Multi-Level Feedback Queue



OSTEP's five-rule MLFQ: three queues with quanta of 4, 8 and 16 minutes, cumulative per-level accounting, and a priority boost every 50 minutes.

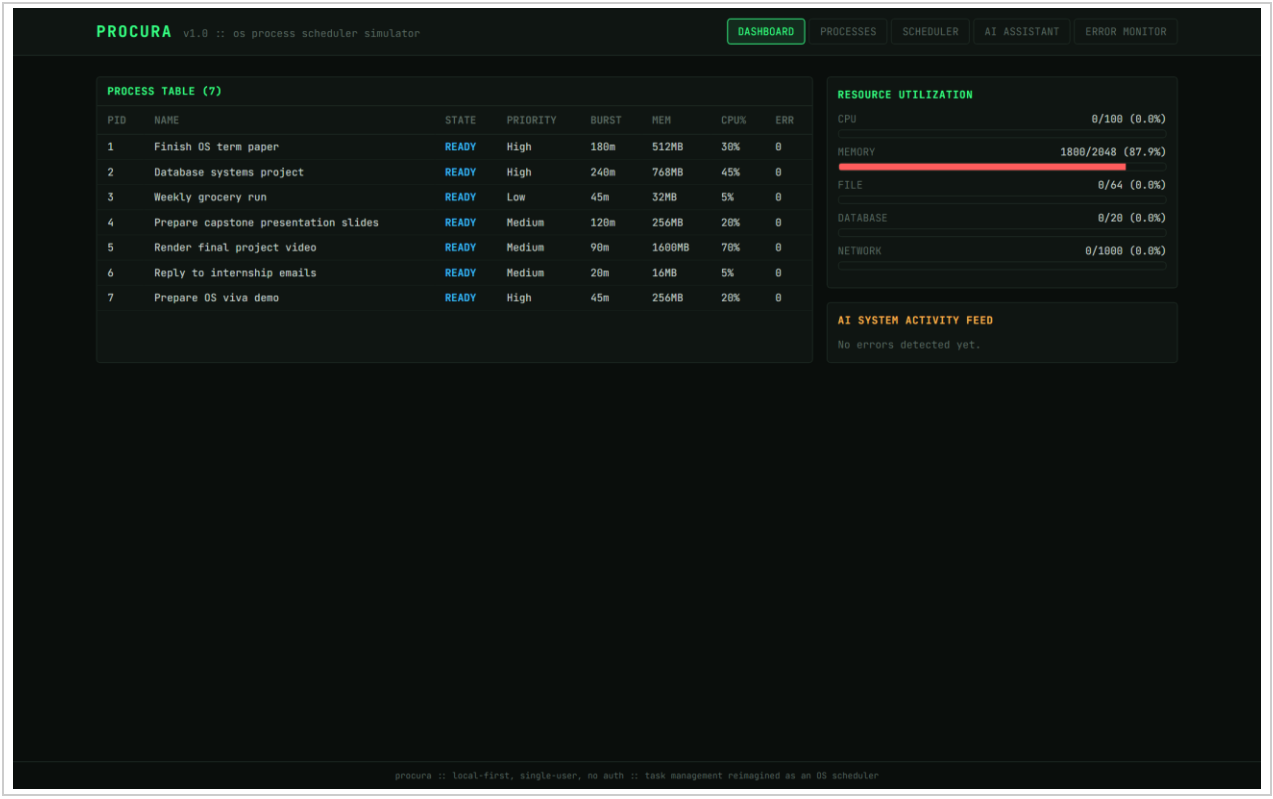
What to notice

- Slices get longer later in the timeline as tasks are demoted into lower queues with larger quanta, so MLFQ needs only 110 slices versus 187 for Round Robin and EEVDF.
- Short tasks finish while still in the high-priority queue: P6 completes at minute 168 (versus 233 under EEVDF).
- Average response time matches Round Robin (11.9 minutes), because every new task enters the top queue.

5. Resource contention and self-healing

This part of the demo shows Procura's detect → diagnose → recover loop, the same architecture used by Kubernetes controllers and AIOps platforms. A memory shortage is created deliberately, the detector finds the resulting allocation failures, the AI explains each one, and a fixed, auditable recovery script decides what to do.

Figure 11. Memory pressure applied

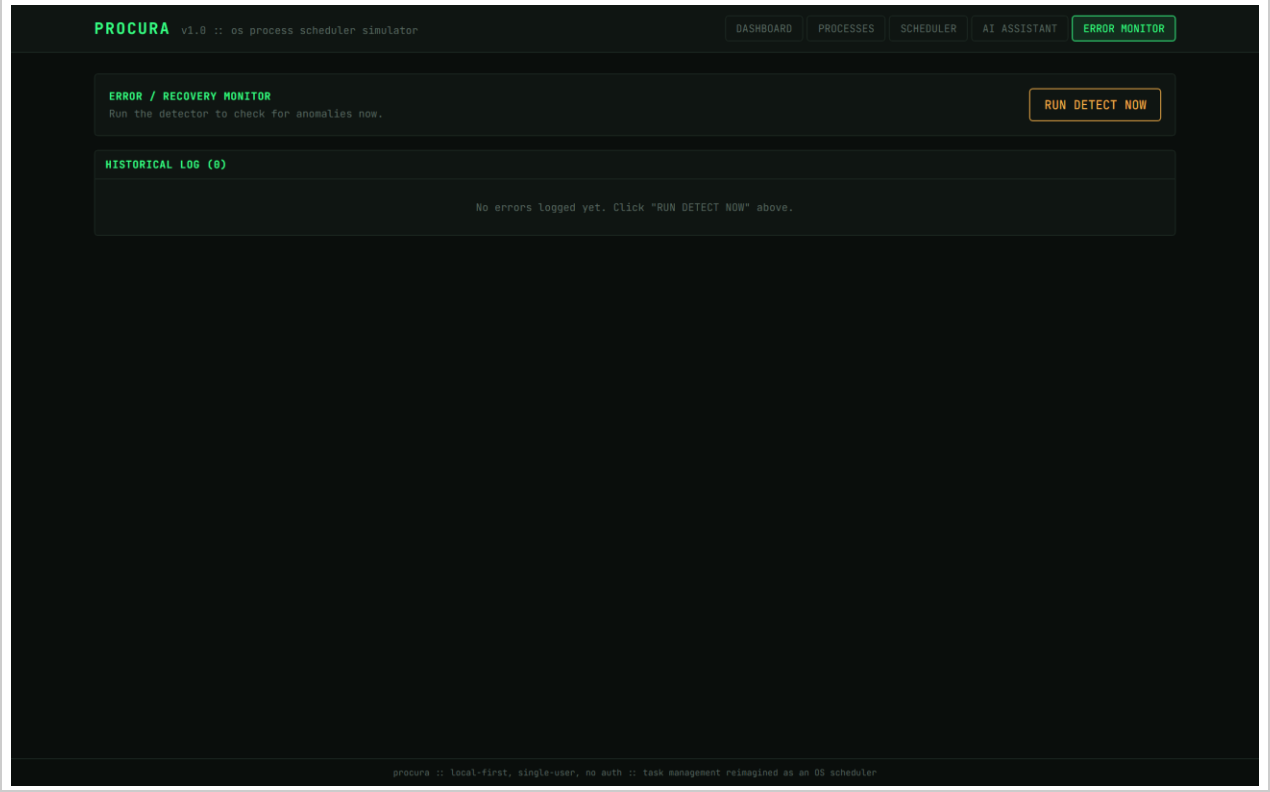


Through the API, the low-priority 'Weekly grocery run' (P3) is given 1200 MB and the term paper (P1) 600 MB.

What to notice

- 1800 of 2048 MB (87.9%) is now in use, leaving 248 MB for every other task.
- The bar turns red above 85% utilisation.

Figure 12. Error Monitor before detection

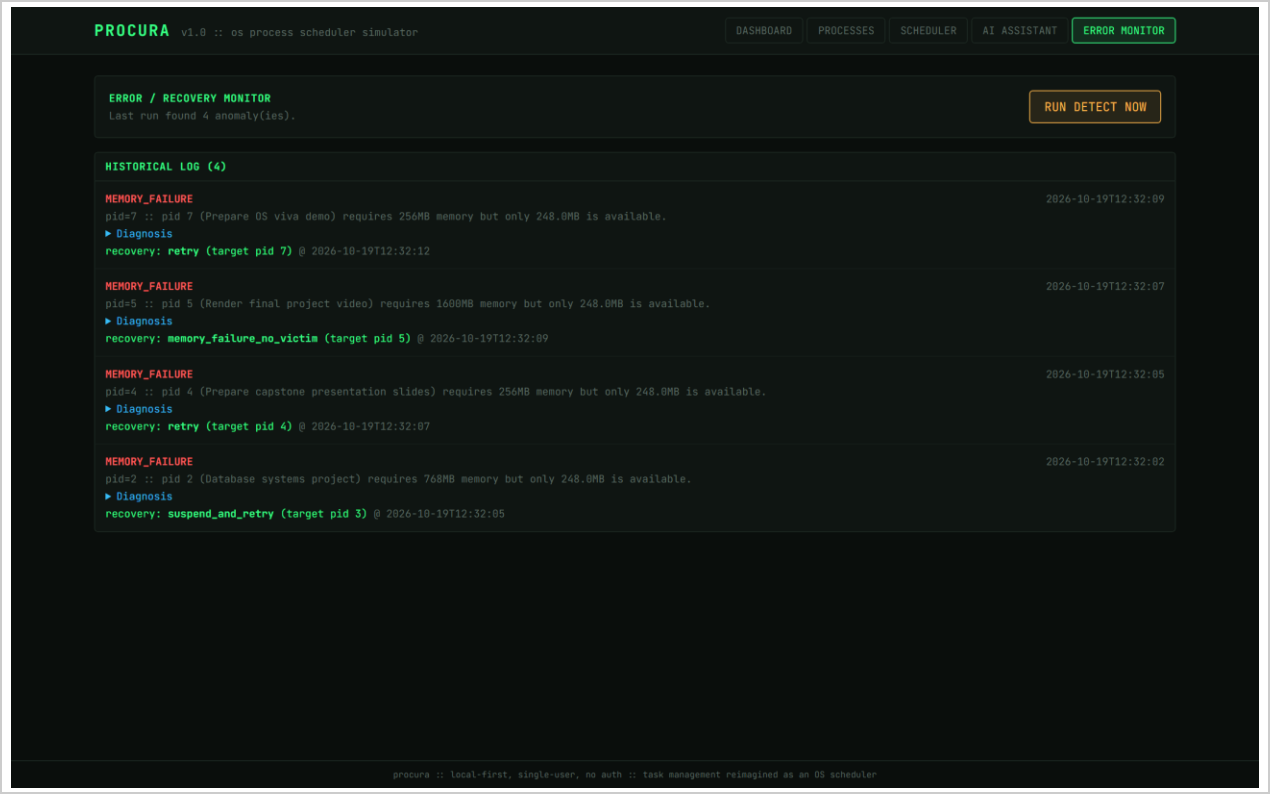


No errors have been recorded yet.

What to notice

- Detection runs on demand through RUN DETECT NOW (POST /detect); the same detector also performs statistical anomaly checks (EWMA by default) and wait-for-graph deadlock detection.

Figure 13. After one detection pass: three kinds of recovery

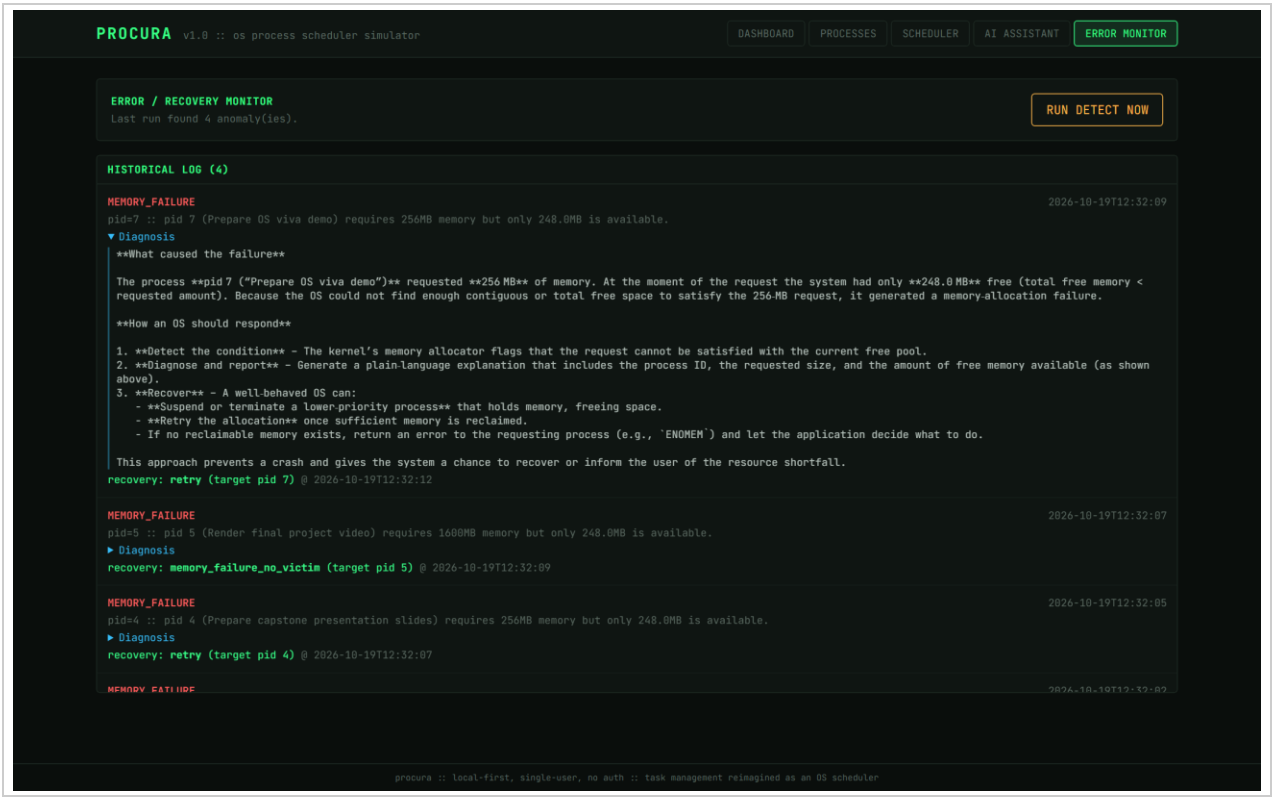


Four memory-allocation failures, each with its diagnosis and the recovery action taken.

What to notice

- P2 (768 MB, High) → suspend_and_retry targeting P3. P3 has strictly lower effective priority and holds 1200 MB, so preempting it actually makes P2's allocation fit.
- P4 and P7 (256 MB each) → retry. Every finding records '248 MB available' because all failures are snapshotted before any recovery runs; by the time their recovery ran, P2's recovery had freed enough memory, so a simple retry succeeded.
- P5 (1600 MB) → memory_failure_no_victim. No lower-priority holder could free enough memory, so P5 is left pending instead of evicting higher-priority work for no benefit.
- These rules follow Kubernetes preemption. They prevent two failure modes seen in an earlier version: two tasks evicting each other repeatedly, and a task being suspended without freeing enough memory to help.

Figure 14. An expanded AI diagnosis

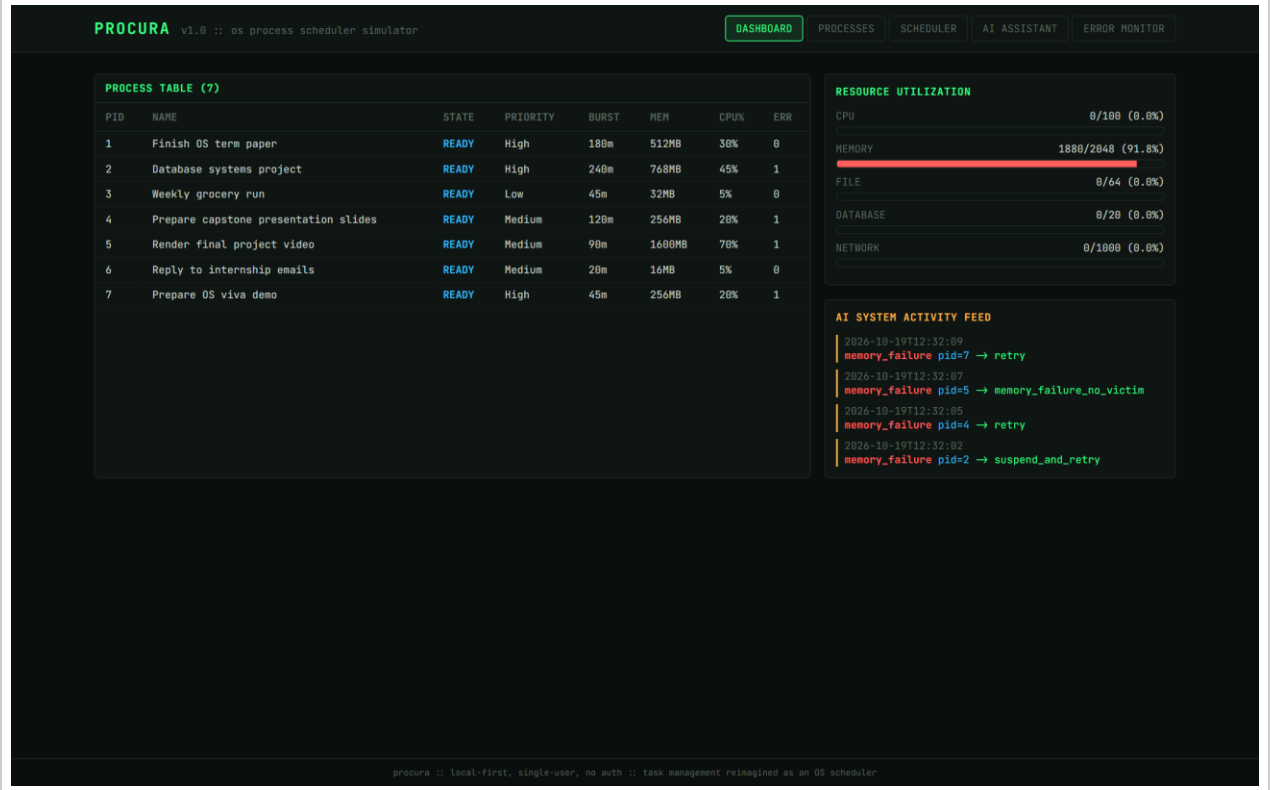


The diagnosis for P7, generated by the RAG assistant.

What to notice

- The explanation cites the recorded facts (256 MB requested, 248 MB free) because the incident details are passed to the model as an authoritative section of its prompt.
- Its general guidance comes from the retrieved Memory_Management and Error_Recovery knowledge-base passages.
- The AI only explains; the recovery decision is made by the fixed script, as in production runbook tools.

Figure 15. Dashboard after recovery



The AI SYSTEM feed lists every detect → recover event, and memory reflects the recovery actions.

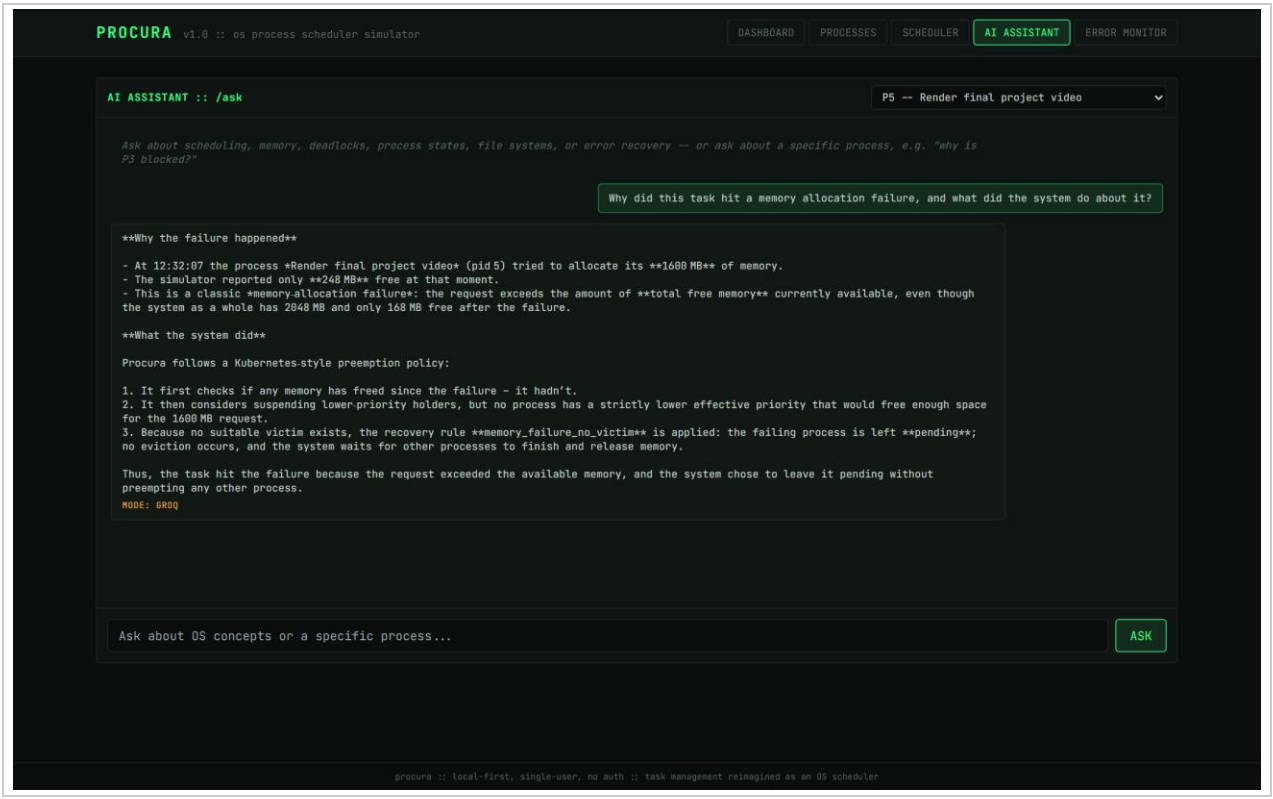
What to notice

- Memory now reads 1880/2048 MB (91.8%): P1 600 + P2 768 + P4 256 + P7 256.
- The ERR column shows 1 for P2, P4, P5 and P7 — the four tasks that failed an allocation.
- P3 still shows READY: suspension released its memory, but the state only changes when the victim was RUNNING. This is a known simplification (Section 8).

6. The AI assistant

The assistant answers questions by retrieving passages from an eight-document operating-systems knowledge base with a TF-IDF index, and, when a process is selected, the recorded incident facts for that process. A configured LLM writes the answer; without an API key, Procura returns a clearly labelled extractive answer built from the retrieved passages.

Figure 16. Grounded answer about a specific task (P5)

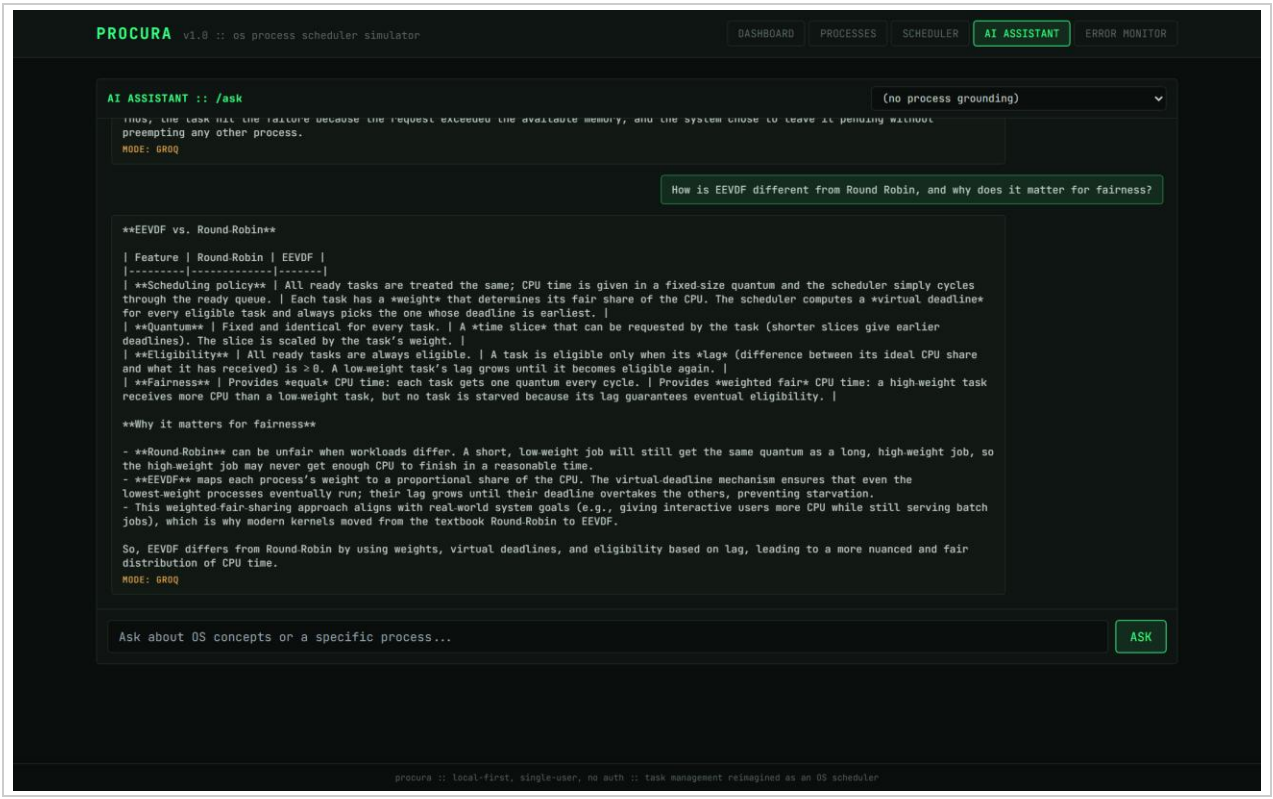


Asked why P5 failed and what the system did, with P5 selected for grounding.

What to notice

- Every fact matches the logs: 1600 MB requested, 248 MB free at the time, memory_failure_no_victim, and 168 MB free afterwards.
- It correctly explains why nothing was evicted: no holder had strictly lower effective priority and would free enough space.
- Before incident grounding was added, the same question produced an invented cause (memory fragmentation). Grounding fixed that, though language models can still add unsupported remarks, so answers should be read alongside the logs.

Figure 17. Concept question: EEVDF versus Round Robin



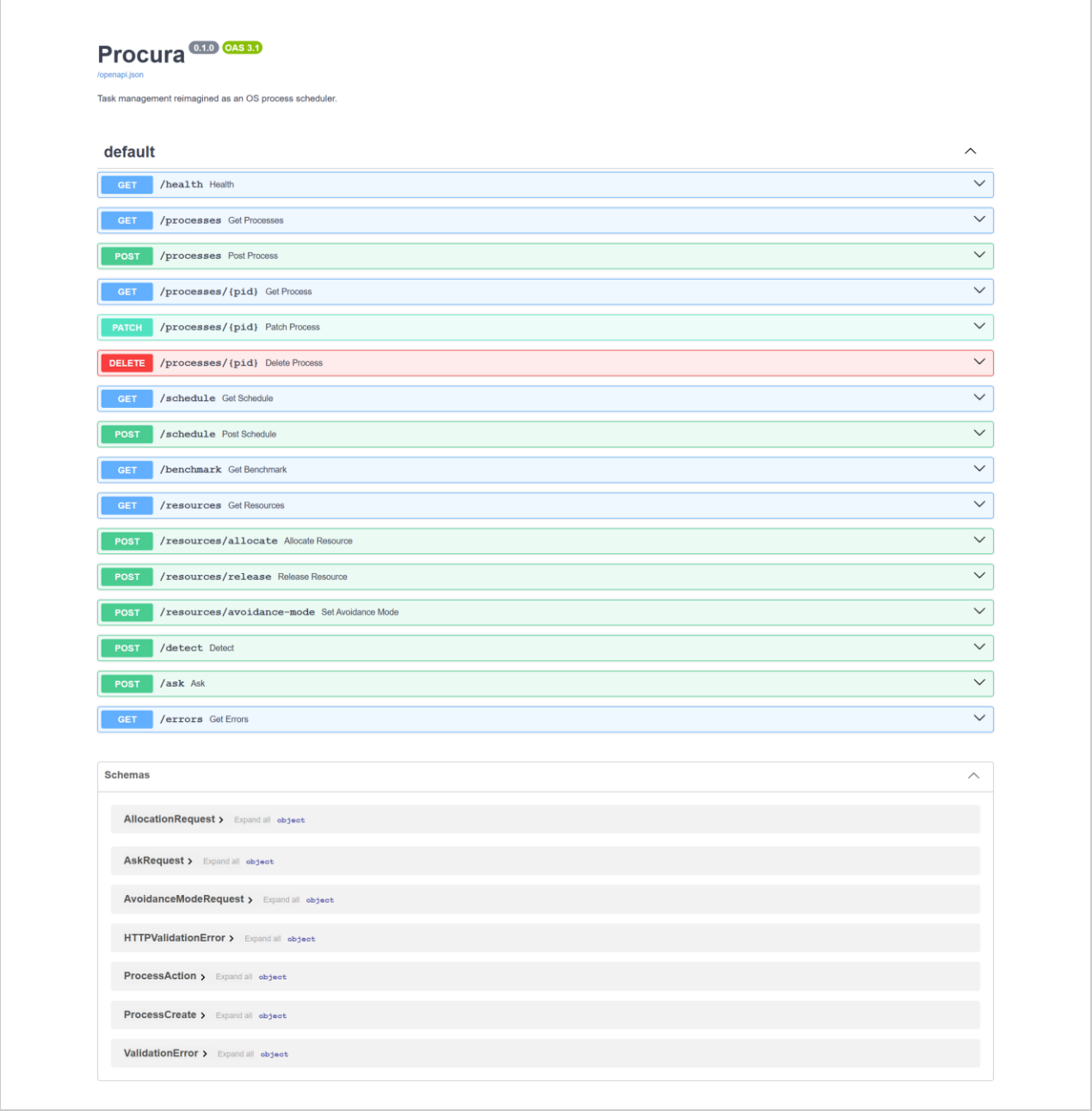
A general question with no task selected, answered from the knowledge base.

What to notice

- The core explanation is correct: weights, lag, eligibility when lag ≥ 0, and dispatch by earliest virtual deadline, with low-weight tasks never starved.
- Before the knowledge base was extended, the model had nothing to retrieve on EEVDF and invented an expansion of the acronym. Seven retrieval tests now guard this coverage.
- Two statements remain overstated: Round Robin does not leave a job unable to finish, and Linux moved to EEVDF from CFS rather than from Round Robin.
- The interface shows the model's markdown raw (asterisks and table pipes); rendering it is listed as future work.

7. API, terminal dashboard, benchmark and tests

Figure 18. Interactive API documentation



FastAPI's auto-generated /docs page.

What to notice

- All 16 REST endpoints are listed: process CRUD and lifecycle, scheduling, benchmarking, resource allocation and avoidance mode, detection, the AI assistant, and the error log.
- Request bodies are validated against the schemas listed at the bottom (e.g. ProcessCreate, AllocationRequest).

Figure 19. procura_top — the terminal dashboard

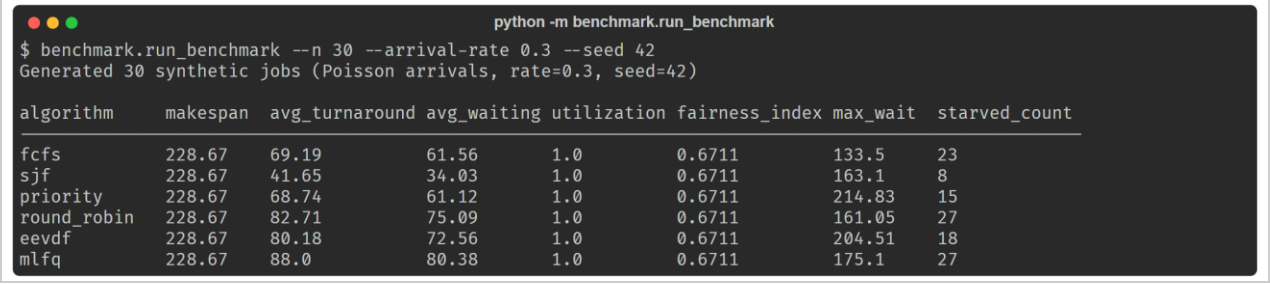


An htop-style view rendered with the rich library, polling the same REST API as the web interface.

What to notice

- The same live state as Figure 15: memory at 91.8% and error counts of 1 for P2, P4, P5 and P7.
- The AI SYSTEM feed shows each event with the start of its diagnosis.
- The memory label is clipped at the panel edge (a cosmetic issue).

Figure 20. Benchmark: all six algorithms on a seeded workload

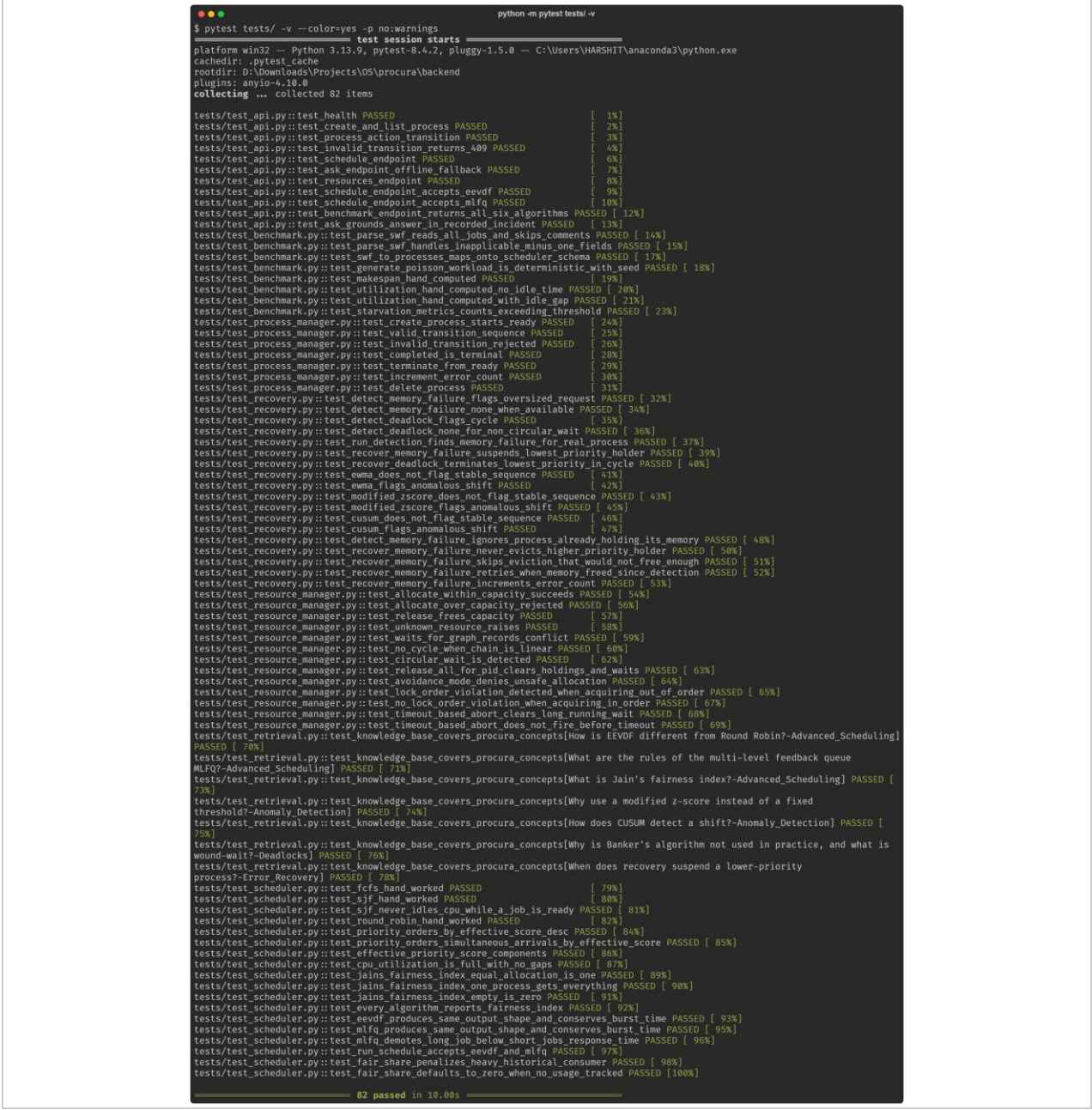


python -m benchmark.run_benchmark --n 30 --arrival-rate 0.3 --seed 42.

What to notice

- 30 synthetic jobs with Poisson arrivals; the seed makes the run exactly reproducible.
- SJF has the lowest average waiting time (34.03) and fewest starved jobs (8), but a higher maximum wait (163.1) than FCFS (133.5).
- Identical makespan and full utilisation confirm that all six algorithms are work-conserving.

Figure 21. Automated test suite



python -m pytest tests/ -v.

What to notice

- 82 tests, all passing: scheduling against hand-worked textbook examples, the state machine, resources and deadlocks, statistical detectors, recovery rules, knowledge-base retrieval, the benchmark module, and API smoke tests.
- Tests never call a live LLM, even when an API key is configured.

8. Observations and known limitations visible in the demo

These are reported because a reviewer looking closely at the screenshots will notice them; none affects the correctness of the scheduling results.

- **Jain's Fairness Index is constant across algorithms.** It is computed over each task's total allocated CPU time, and every algorithm eventually gives every task exactly its burst time. The index therefore measures how much each task gets, not when; waiting and response time show the real differences.
- **AI answers can still overstate.** Grounding in incident facts and an extended knowledge base removed the invented causes seen earlier, but Figure 17 still contains two overstated claims.
- **Raw markdown in AI answers.** The interface does not render the model's markdown formatting.
- **Suspended tasks keep the READY state.** Suspension releases the victim's memory, but the state changes only if the victim was RUNNING.
- **Seed deadlines have passed.** The demo tasks' deadlines fall in September 2026, before the capture date, so four tasks receive the maximum urgency score. This shapes the Priority ordering in Figure 7.
- **Clipped memory label in procura_top.** A cosmetic layout issue in the terminal panel.